

02 — gp_velocity Module Design

What you'll learn: the full design of `dvl_correction/src/gp_velocity.py` — config, API, kernel, windowing, refit, and dropout-bridge logic.

Prereqs: [references/02-03](#),
[01-codebase-architecture-and-seams](#).

Why it matters here: this is the buildable spec for the new module.

1. Public API (engine-agnostic)

The module is designed so the **engine** (GPyTorch now, state-space Kalman later) hides behind one interface. Pipeline code only ever calls these:

```
@dataclass(frozen=True)
class GpVelocityConfig:
    window_seconds: float = 120.0      # sliding-window length of DVL history
    min_samples: int = 12               # below this, passthrough (no GP)
    refit_every: int = 10               # refit hyperparameters every N ingests
    bridge_after_s: float = 1.5         # DVL silence before we start bridging ( $\approx >1$  DVL period)
    bridge_cadence_s: float = 1.0      # how often to inject a bridge update during silence
    max_bridge_s: float = 20.0          # stop bridging after this (let IMU take over fully)
    outlier_mahalanobis: float = 9.0    # GP pre-filter threshold ( $3\sigma$  in 3-D  $\approx 9$ )
    jitter: float = 1e-6
    # kernel/init hyperparameters (seeds; learned by fit)
    init_lengthscales: float = 5.0
    init_noise_std_m_s: float = 0.20    # ~ KalmanConfig.default_dvl_velocity_std_m_s

class GpDvlVelocityModel:
    def __init__(self, config: GpVelocityConfig | None = None) -> None: ...

    def ingest(self, timestamp_s: float,
               velocity_body_m_s, covariance_body=None) -> None:
        """Append an accepted DVL velocity to the sliding window."""

    def fit(self) -> None:
        """(Re)optimize hyperparameters on the current window (called internally per refit_every)."""

    def predict(self, timestamp_s: float) -> tuple[np.ndarray, np.ndarray]:
        """Return (mean3, cov3x3) body-frame velocity at a query time.
        Variance grows with extrapolation distance from the window."""

    def is_outlier(self, timestamp_s: float, velocity_body_m_s) -> bool:
        """Mahalanobis distance of a candidate vs the GP predictive band > threshold."""

    @property
    def ready(self) -> bool:
        """True once  $\geq$  min_samples are in the window (else callers should passthrough)."""

    @property
    def last_ingest_s(self) -> float | None: ...
```

Return types are plain numpy (mean3 shape (3,), cov3x3 shape (3,3), SPD) so the pipeline

can

build a `CorrectedDvlMeasurement` directly — no torch types leak out.

2. Internal state

- A ring buffer of `(t, v_body[3], per_sample_var[3] | None)` trimmed to `window_seconds`.
- Cached fitted model/hyperparameters; an ingest counter for `refit_every`.
- **Lazy imports:** `import torch, gpytorch` happen inside methods, so importing `dvl_correction` without the `[gp]` extra never fails. If unavailable and the model is used, raise a clear error pointing at `pip install -e ./dvl_correction[gp]`.

3. The model (GPyTorch, multitask)

- **Inputs:** time, **centered** on the window start and scaled (numerical conditioning, references/03 §7).
- **Outputs:** 3-D body-frame velocity, **standardized** per axis (undo on output).
- **Kernel:** ``MultitaskKernel(base = ScaleKernel(MaternKernel(v=1.5)) + ScaleKernel(RQKernel())``
 - `ScaleKernel(RBFKernel()), num_tasks=3, rank=...)` — the composite from references/02 §7-8: Matérn-3/2 (physical smoothness) + RQ (multi-scale) + RBF (trend), with a learned 3×3 task covariance for cross-axis correlation.
- **Likelihood:** `MultitaskGaussianLikelihood`; when per-sample covariance_body is supplied, use a **fixed/added per-point noise** so noisier DVL samples are trusted less (references/03 §4).
- **Training (`fit`):** maximize the exact marginal likelihood with Adam for a small fixed number of iterations on the window; warm-start from the previous fit's hyperparameters for speed/stability.

Lighter fallback (if torch is unwanted): three independent `sklearn.gaussian_process.GaussianProcessRegressors` (one per axis) with `Matern(v=1.5)+RationalQuadratic+RBF, return_cov` for the diagonal. Loses cross-axis correlation but keeps the same external API.

4. predict() semantics (the important behaviours)

- **Inside the window / near data:** posterior mean denoises; covariance is small.
- **Short extrapolation (a dropout just started):** mean continues the trend; covariance modestly inflated.
- **Long extrapolation:** covariance climbs toward the prior, so downstream weighting collapses gracefully. This is automatic from the GP math — we don't hand-code variance inflation.
- Always returns an **SPD** cov3x3 (add `jitter*I`, symmetrize).

5. Dropout-bridge logic (where modes combine)

Pseudo-logic the pipeline drives (see 01 §3 Seam C):

```
on accepted DVL sample at t, v:
    if model.ready and not model.is_outlier(t, v):
        m, C = model.predict(t)          # denoise
        emit CorrectedDvlMeasurement(t, velocity_body_m_s=m, covariance_body=C)  # adaptive R
        model.ingest(t, v, meas_cov)
    elif model.ready and model.is_outlier(t, v):
        diagnostics.gp_outlier_rejections += 1    # drop; EKF gate is backstop
    else:
        emit the raw corrected measurement unchanged  # passthrough until min_samples
        model.ingest(t, v, meas_cov)

during DVL silence (driven in _propagate_filter_to):
    gap = now - model.last_ingest_s
    if model.ready and bridge_after_s < gap <= max_bridge_s, every bridge_cadence_s:
        m, C = model.predict(now)          # C grows with gap
        emit CorrectedDvlMeasurement(now, velocity_body_m_s=m, covariance_body=C)
        diagnostics.gp_bridge_updates += 1
    # gap > max_bridge_s: stop bridging, let IMU dead-reckoning run
```

Note: bridge measurements are **not** re-ingested (they are model outputs, not observations) — avoids the GP feeding on its own predictions.

6. Refit cadence & cost

- Refit on the window every `refit_every` ingests (not every sample): hyperparameters move slowly (references/03 §8).
- With `window_seconds=120` at ~ 1 Hz, $n \approx 120$, multitask 360×360 — a few ms per refit on a laptop.

Predict is sub-ms. Real-time-trivial for the demo (references/08 §5).

7. Designed for the edge swap

`predict()/ingest()` is exactly the interface a **state-space Matérn GP** implements (references/05). The edge version replaces the GPyTorch internals with a tiny Kalman recursion (state $\approx 2/\text{axis}$), keeps the same external behaviour, drops torch, and can take its process noise from imu-drift's Allan-variance fit. No pipeline changes when swapping. Roadmap: 04-edge-and-spde-roadmap.

8. Edge cases to handle

- Fewer than `min_samples` → `ready=False`, passthrough.
- Duplicate/near-equal timestamps → drop the duplicate before fit (jitter won't save a singular K).
- All-window-outliers → fit on what's left; rely on EKF gate; consider a robust loss later.
- NaN/inf guard on inputs and on predict outputs (return passthrough + log if the fit is degenerate).

Key takeaways

- One class, engine-agnostic ingest/fit/predict/is_outlier, numpy in/out, lazy torch import.
- Multitask **Matérn-3/2 + RQ + RBF** kernel over centered time → standardized 3-D velocity, with per-sample noise.
 - Denoise + adaptive R + outlier pre-filter + **gap bridging with auto-growing variance**, capped by `max_bridge_s`; bridge outputs are not re-ingested.
 - Same interface enables the **$O(n)$ edge swap** with zero pipeline change.

Further reading

- 03 — Benchmark and frontend demo
- GPyTorch multitask GP regression tutorial (gpytorch.ai).